

Dashboard IoT Telemetry Platform

Memoria Técnica

Grupo 3



Alumnos desarrolladores:

Eduardo García Gómez Cabrero

Rebeca Conde Acero

Javier Sánchez Cañaveras

Pablo Muñoz Pedrero

Tutor de prácticas empresa:

David Manzanares Muñoz

Tutor de prácticas Instituto Nebrija:

José Carlos Rodríguez de la Llana

Índice

Índice.....	1
1. Introducción.....	2
2. Objetivos del Proyecto.....	2
3. Tecnologías Utilizadas.....	2
4. Arquitectura del Sistema.....	2
Simulador.....	3
Backend.....	3
Base de Datos.....	3
Visualización.....	3
Esquema sobre la arquitectura del sistema.....	4
5. Estructura del Proyecto.....	4
6. Decisiones Técnicas Adoptadas.....	5
Uso de FastAPI.....	5
PostgreSQL como Base de Datos.....	5
Generación Automática de Alarmas.....	6
Uso de Grafana.....	6
Contenerización con Docker.....	6
7. Implementación de Docker.....	6
8. Funcionalidades Implementadas.....	7
Gestión de Telemetría.....	8
Gestión de Dispositivos.....	8
Gestión de Alarmas.....	8
Gestión de Eventos de Báscula.....	8
Indicadores KPI.....	8
9. Resultados Obtenidos.....	8
10. Posibles mejoras futuras.....	8
11. Conclusiones.....	9

1. Introducción

El presente proyecto consiste en el desarrollo de una plataforma de monitorización y telemetría IoT orientada a un entorno industrial. El sistema permite la recepción, almacenamiento, procesamiento y visualización de datos procedentes de dispositivos simulados que representan secaderos y básculas industriales.

El software ha sido desarrollado utilizando una arquitectura basada en servicios desacoplados, permitiendo la escalabilidad y facilitando futuras ampliaciones del sistema.

2. Objetivos del Proyecto

Los principales objetivos definidos para el proyecto han sido:

- Diseñar una arquitectura capaz de recibir datos de dispositivos IoT.
- Crear un simulador para la generación de los datos sobre secaderos y básculas.
- Almacenar la información de forma persistente en una base de datos relacional.
- Implementar una API REST para la gestión y consulta de datos.
- Generar alarmas automáticas a partir de determinados umbrales.
- Crear dashboards visuales para el análisis de la información.
- Facilitar el despliegue reproducible mediante contenedores Docker.

3. Tecnologías Utilizadas

Tecnología	Función
Python	Lenguaje principal del proyecto
FastAPI	Desarrollo de la API REST
PostgreSQL	Persistencia de datos
psycopg2	Conector PostgreSQL para Python
Uvicorn	Servidor ASGI
Requests	Comunicación HTTP desde el simulador
Grafana	Visualización y análisis de datos
Docker / Docker Compose	Contenerización de servicios / Orquestación de contenedores

4. Arquitectura del Sistema

La arquitectura implementada sigue una estructura en capas compuesta por los siguientes elementos:

Simulador IoT → Backend FastAPI → PostgreSQL → Grafana

Simulador

El simulador genera datos que representan el comportamiento de dispositivos industriales:

- Secaderos industriales.
- Básculas industriales.

Estos datos son enviados mediante peticiones HTTP a la API REST del sistema.

Backend

El backend ha sido desarrollado utilizando FastAPI y actúa como núcleo del sistema.

Sus responsabilidades principales son:

- Recepción de datos de telemetría.
- Validación de la información recibida.
- Inserción de registros en la base de datos.
- Consulta de datos históricos.
- Generación automática de alarmas.
- Cálculo de indicadores KPI.

Base de Datos

Se ha utilizado PostgreSQL como sistema gestor de bases de datos debido a su robustez, estabilidad, compatibilidad con aplicaciones de análisis de datos y el sistema de gestión empresarial de Odoo.

Las tablas principales son:

- devices
- telemetry
- alarms
- scale_events

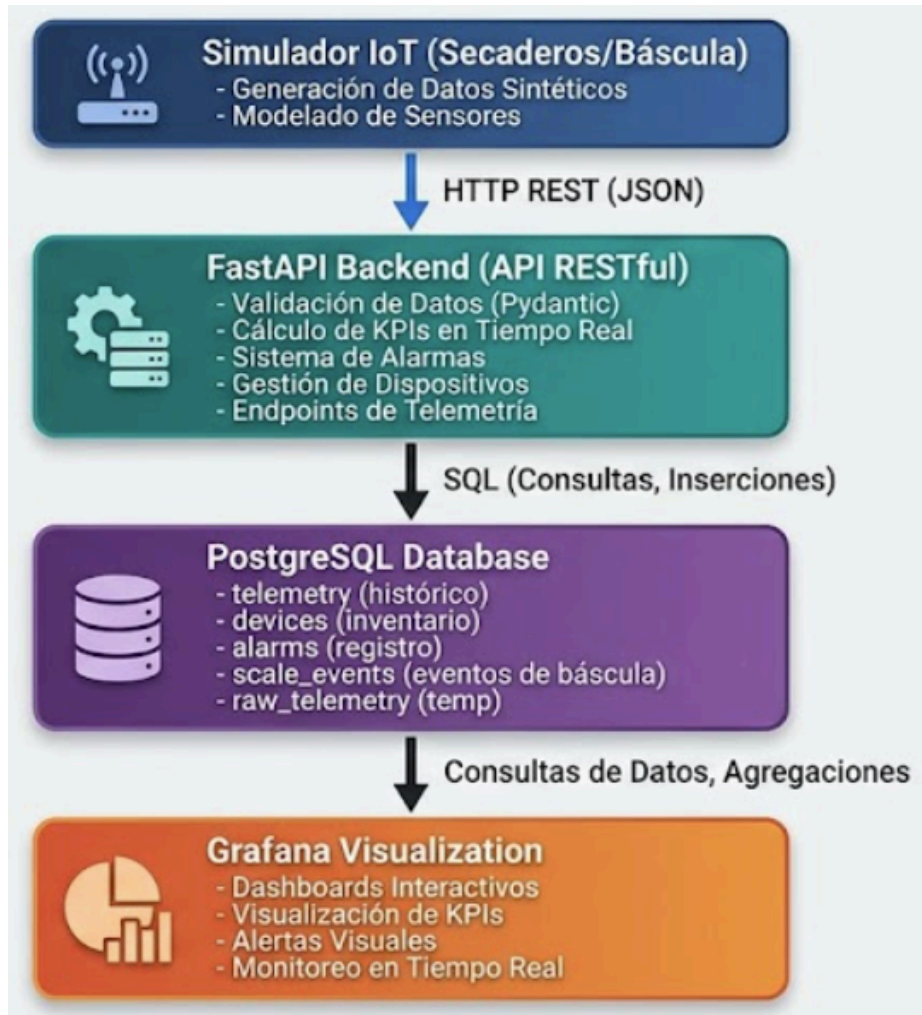
Visualización

Grafana se conecta directamente a PostgreSQL para mostrar la información mediante dashboards interactivos.

Actualmente se dispone de:

- Dashboard de secaderos.
- Dashboard de básculas.
- Dashboard de KPIs generales.

Esquema sobre la arquitectura del sistema



5. Estructura del Proyecto

La organización del proyecto sigue una estructura modular para facilitar el mantenimiento y escalabilidad:

```
iot-telemetry-project/
├── app/
│   ├── core/
│   ├── models/
│   ├── routes/
│   └── services/
├── db/
│   └── iot_db_backup.sql
├── grafana/
│   ├── dashboard-dryers.json
│   ├── dashboard-scales.json
│   ├── dashboard-summary-kpis.json
│   └── provisioning/
├── iot-telemetry-project-simulator/
│   └── src/
├── doquer-compose.yml
├── Dockerfile
├── requirements.txt
├── main.py
├── README.md
├── Instrucciones de instalación.pdf
└── Memoria técnica Proyecto Dashboard iot telemetría.pdf
```

La separación por módulos permite aislar la lógica de negocio, los modelos de datos y las rutas de la API.

6. Decisiones Técnicas Adoptadas

Uso de FastAPI

Se seleccionó FastAPI por ofrecer:

- Alto rendimiento.
- Documentación automática mediante Swagger.
- Validación automática de datos.
- Facilidad para construir APIs REST.

PostgreSQL como Base de Datos

La elección de PostgreSQL se realizó por:

- Software libre.
- Robustez y estabilidad.
- Fiabilidad en entornos productivos.
- Buen rendimiento con grandes volúmenes de datos.
- Compatibilidad con herramientas de análisis como Grafana.
- integración con Odoo.

Generación Automática de Alarmas

Se implementó un sistema de alarmas basado en reglas de negocio:

- Temperatura superior a 28 °C.
- Humedad superior a 65 %.

Esto permite detectar condiciones potencialmente anómalas sin intervención manual.

Uso de Grafana

Grafana proporciona:

- Visualización en tiempo real sintéticos (generados por el simulador).
- Filtros temporales.
- Creación rápida de paneles analíticos: Facilita la creación de dashboards sin desarrollar un frontend específico.
- Integración nativa con PostgreSQL.

Contenerización con Docker

Como mejora de despliegue se incorporó una rama específica con soporte Docker.

La solución utiliza tres contenedores:

1. PostgreSQL.
2. Backend FastAPI.
3. Grafana.

Esta decisión permite:

- Eliminar dependencias del entorno local.
- Despliegues reproducibles.
- Configuración homogénea en distintos entornos.
- Simplificación de la instalación del proyecto.
- Simplificar futuros despliegues.

7. Implementación de Docker

Para facilitar la puesta en marcha del proyecto se desarrolló una configuración basada en Docker Compose.

Se añadieron los siguientes archivos:

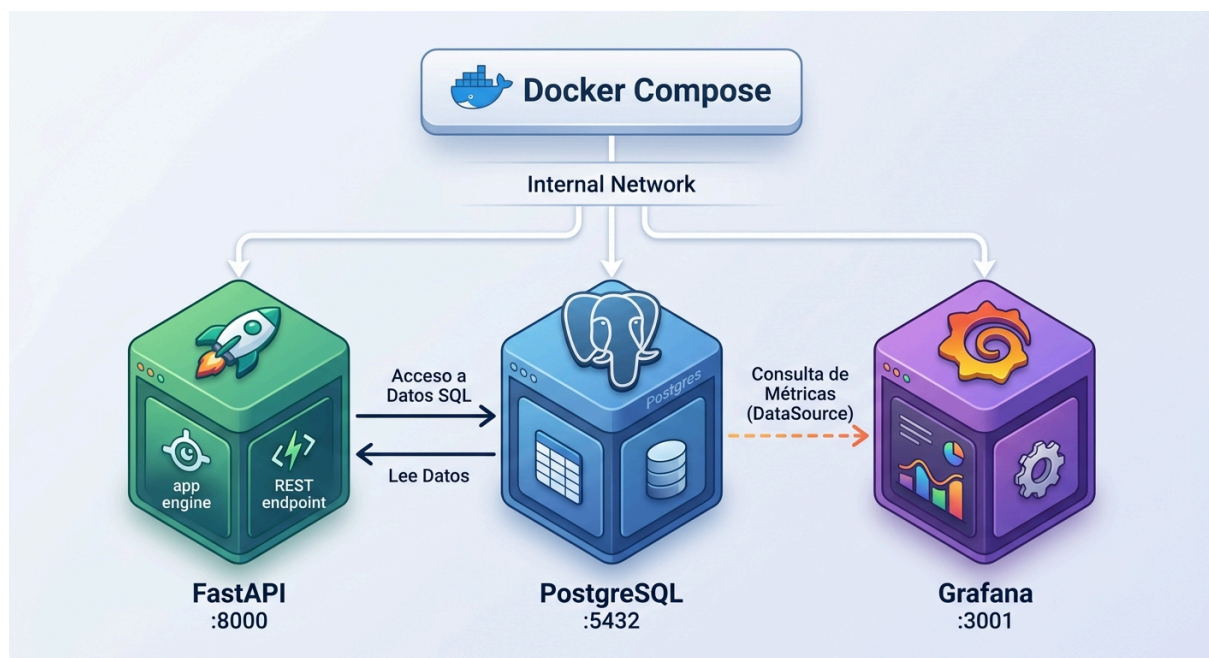
- Dockerfile
- docker-compose.yml
- .dockerignore
- requirements.txt
- grafana/provisioning/
- app/core/database.py
- README.md

La configuración permite iniciar automáticamente:

- PostgreSQL con la base de datos `iot_db`.
- FastAPI en el puerto 8000.
- Grafana en el puerto 3001.

Además, Grafana carga automáticamente:

- La fuente de datos PostgreSQL.
- Los dashboards exportados.
- La estructura de carpetas del proyecto.



8. Funcionalidades Implementadas

Actualmente el sistema dispone de:

Gestión de Telemetría

- Inserción de datos.
- Consulta de registros.
- Filtrado por dispositivo.

Gestión de Dispositivos

- Registro de dispositivos.
- Consulta de dispositivos existentes.

Gestión de Alarmas

- Generación automática.
- Consulta de incidencias.

Gestión de Eventos de Báscula

- Inserción de eventos.
- Consulta histórica.

Indicadores KPI

El endpoint /kpis/ proporciona:

- Número total de mediciones.
- Número total de eventos de báscula.
- Número total de alarmas.
- Temperatura media.
- Humedad media.
- Última telemetría registrada.

9. Resultados Obtenidos

Las pruebas realizadas muestran que:

- El backend recibe correctamente la información enviada por el simulador.
- PostgreSQL almacena los datos de forma persistente.
- Las alarmas se generan automáticamente cuando se superan los umbrales definidos.
- Los dashboards muestran la información correctamente.
- El despliegue mediante Docker funciona de forma satisfactoria.
- La documentación Swagger/OpenAPI facilita la validación y prueba de los endpoints.

10. Posibles mejoras futuras

- Implementación de autenticación mediante JWT (JSON Web Token).
- Gestión de usuarios y roles.
- Comunicación MQTT para dispositivos IoT reales.
- Despliegue en la nube.
- Alertas por correo electrónico.
- Integración con sistemas SCADA industriales.
- Incorporación de métricas históricas avanzadas.

11. Conclusiones

Se ha desarrollado una plataforma completa de telemetría IoT capaz de simular un entorno industrial real. El software integra generación de datos, almacenamiento, procesamiento y visualización mediante herramientas ampliamente utilizadas en el ámbito profesional.

La incorporación de Docker mejora significativamente la portabilidad y facilita el despliegue del sistema en distintos entornos, permitiendo una futura evolución hacia escenarios más próximos a producción.